Step 1: Build the UI Foundation

Enter this prompt to build the basic user interface:

Build a React application using Vite, TypeScript, and Tailwind CSS that creates an "Interactive Alert System" UI. Focus only on the visual interface - no backend functionality yet.

Application Layout:
- Full screen gradient background: `bg-gradient-to-br from-blue-500 to-violet-600`
- Centered container with padding

Main Components:

1. **Title Section:**
   - "Interactive Alert System" as main heading
   - Descriptive subtitle below

2. **"Ask Anything" Search Bar:**
   - Glassmorphic container with animated gradient background
   - Input field with amber "Ask" button
   - Include loading states (spinner) but no actual functionality

3. **Six Alert Cards in a Grid:**
   - 3 columns on desktop, 1 on mobile
   - Each card has glassmorphic styling
   - Cards include:
     - Tech Updates (Globe icon)
     - Breaking News (AlertCircle icon)
     - Content Analysis (BarChart2 icon)
     - Historical Events (Clock icon)
     - Memes & Trends (MessageSquare icon)
     - Community Pulse (Bell icon)
   - Each card has its own input field and button
   - Include loading states but no functionality

4. **Floating Action Button:**
   - Fixed bottom-right position
   - Plus icon

5. **Response Modal (Hidden by default):**
   - Use @radix-ui/react-dialog
   - Dark glassmorphic design
   - Markdown rendering capability with react-markdown

Create placeholder state management but no API calls. Make it visually stunning with smooth animations and hover effects.

Step 2: Set Up Supabase

Manual Steps:

1. Create a Supabase account at https://supabase.com
2. Create a new project
3. Connect it using the authentication button in the top-right of the interface
4. Keep this tab open - you'll need it for the next step

Step 3: Deploy Supabase Edge Function

Once Supabase is connected, enter this prompt:

Now let's add the Supabase Edge Function to handle API requests. Deploy a new edge function called 'webhook-proxy' that will:

1. Accept POST requests from the React app
2. Forward them to an n8n webhook URL (which we'll configure next)
3. Handle CORS properly
4. Return responses back to the React app

The edge function should:
- Accept a request body with: type, prompt, question, and metadata
- Add proper CORS headers
- Forward to a placeholder n8n URL for now (we'll update it later)
- Handle errors gracefully

After deploying, update the React app to use this edge function URL for all API calls but keep the n8n webhook URL as a placeholder for now.

Step 4: Set Up n8n Workflow

Manual Steps:

1. Create an account at https://n8n.io or self-host n8n
2. Import your n8n workflow JSON
3. Find the webhook node in your workflow
4. Copy the **Production** webhook URL (not the test URL!)

5. Make sure your workflow is active

Step 5: Connect Everything Together

Enter this final prompt with your n8n webhook URL:

Final step! Update the Supabase edge function to use this n8n webhook URL: [PASTE YOUR N8N WEBHOOK URL HERE]

Then ensure:
1. The edge function forwards all requests to this n8n webhook
2. The React app properly displays responses in the modal
3. Error handling works correctly
4. The response formatting function properly extracts and cleans the n8n output

Test that:
- General "Ask Anything" questions work
- Each of the 6 alert cards sends proper requests with their specific prompts
- Responses are displayed in the modal with proper markdown formatting
- Loading states work correctly
- Error messages appear for failed requests

Important Notes:

- **Always use the Production webhook URL** from n8n, not the test URL
- Make sure your n8n workflow is **Active** before testing
- The edge function acts as a proxy to avoid CORS issues
- Each alert card sends its specific prompt along with the user's question
- The response modal should handle various response formats from n8n

Troubleshooting:

- If requests fail, check the Supabase function logs
- Verify the n8n webhook URL is correct and the workflow is active
- Check browser console for any CORS errors
- Ensure the edge function has proper error handling